PRÁCTICA DE LABORATORIO: LIBRERÍAS EN PYTHON

Módulos Estándar: os, sys y platform — Trabajo Práctico de Aula

Objetivo de la Práctica: Ejercitar el uso de librerías incorporadas de Python para interactuar con el sistema operativo, consultar el entorno del intérprete e inspeccionar directorios.

Estilo y Buenas Prácticas (PEP 8): Usar convención *snake_case* para nombres de variables y funciones. Importar siempre las librerías necesarias al inicio del archivo (import os, import sys, import platform).

Metodología Polya: En cada problema, identificar la función específica de la librería requerida, planificar la estructura del script y verificar el resultado por pantalla.

SECUENCIA DE PROBLEMAS PRÁCTICOS EN AULA

PROBLEMA 1: Inspección de Rutas y Verificación de Directorios (Módulo os)

El administrador del sistema del Laboratorio de Informática del CURZAS necesita verificar la ubicación de trabajo actual y comprobar la existencia de carpetas clave del sistema operativo.
Consigna: Escribir un script en Python que realice las siguientes acciones:
a) Obtener e imprimir en pantalla la ruta del directorio de trabajo actual utilizando la función os.getcwd().
b) Comprobar si existe la ruta del directorio '/home' en el sistema utilizando la función os.path.exists(path) e informar el resultado con un mensaje claro (ej. *'El directorio existe'* o *'El directorio no existe'*).

Fase Polya / Guía de Resolución:

1. ¿Qué librería de Python permite interactuar con el sistema operativo y consultar rutas?
2. Escribir el comando necesario para importar la librería al inicio del script.
3. Espacio para boceto de código:

PROBLEMA 2: Análisis de la Versión del Intérprete de Python (Módulo sys)

En la auditoría de software de los servidores de la Tecnicatura (TUASySL) se requiere extraer y procesar la información del intérprete de Python en uso.
Consigna: Escribir un programa en Python utilizando el módulo sys para:
a) Obtener e imprimir la cadena completa con el número de versión mediante sys.version.
b) Mostrar en pantalla la longitud total de la cadena de texto obtenida utilizando la función len().
c) Convertir dicha cadena en una lista de palabras/elementos aplicando el método de strings .split().
d) Imprimir por separado el primer elemento (índice 0) y el último elemento (utilizando indización negativa [-1] o el tamaño de la lista).

Fase Polya / Guía de Resolución:

1. ¿Qué atributo del módulo sys contiene la versión del intérprete?
2. ¿Qué método de cadenas permite separar una frase en palabras individuales para formar una lista?
3. Espacio para boceto de código:

PROBLEMA 3: Detección Automática del Sistema Operativo (Módulo platform)

Para garantizar la compatibilidad de los scripts desarrollados en el CURZAS, se necesita detectar dinámicamente si el entorno de ejecución es un sistema Linux o Windows.
Consigna: Desarrollar un script en Python utilizando el módulo platform que:
a) Obtenga los datos de la plataforma del sistema mediante la función platform.uname().
b) Imprima por pantalla el nombre del Sistema Operativo detectado (accediendo al primer elemento de la tupla devuelta con platform.uname()[0]).
c) Recorra todos los campos devueltos por platform.uname() mediante un ciclo for e imprima cada dato en una columna separada.

Fase Polya / Guía de Resolución:

1. ¿Qué función de la librería platform devuelve los datos del sistema operativo?
2. ¿Cómo se accede al primer valor de la tupla resultante de platform.uname()?
3. Espacio para boceto de código:

PROBLEMA 4: Inspección de Archivos y Conteo de Scripts Python (Función + Bucle)

Como proyecto integrador del tema librerías, se requiere crear un script modular que analice los archivos de un directorio y contabilice cuántos scripts de Python (extensión .py) contiene.
Consigna: Escribir un programa modular en Python que incluya:
a) Una función llamada contar_archivos_py(lista_archivos) que reciba una lista de nombres de archivos, recorra la lista mediante un ciclo for, analice mediante rebanado de cadenas si los últimos 3 caracteres son '.py' (if nombre[-3:] == '.py':) y devuelva el total acumulado con return.
b) En el bloque principal del programa: obtener la ruta actual con os.getcwd(), listar todos los elementos con os.listdir(), invocar a la función definida e imprimir en pantalla la ruta analizada y la cantidad total de scripts .py encontrados.

Fase Polya / Guía de Resolución:

1. ¿Qué función del módulo os devuelve una lista con todos los archivos de una carpeta?
2. ¿Cómo se extraen los últimos 3 caracteres de un string usando slicing o rebanado?
3. Espacio para desarrollo de la función y bloque principal: